

ĐẠI HỌC QUỐC GIA TP.HCM
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



ĐỒ ÁN TỔNG HỢP - HƯỚNG
CÔNG NGHỆ PHẦN MỀM
(CO3103)

BÁO CÁO ĐỒ ÁN
HỆ THỐNG CHIA SẺ TẬP TIN
THÔNG QUA TELEGRAM

Nhóm: 01 - Học kỳ: 251

GVHD: Th.S Lê Đình Thuận

Thành viên: Nguyễn Nguyên Ngọc (2352814)
Bùi Hoàng Cung (2310376)
Nguyễn Trí Thành (2313127)
Võ Hùng Dũng (2310577)



HCMC, December 13th, 2025

Danh sách thành viên và phân công nhiệm vụ

STT	Họ tên	MSSV	Lớp	Nhiệm vụ	Hoàn thành
1	Nguyễn Nguyễn Ngọc	2352814	CC01	– Tổng hợp và tạo file sườn – Kiểm tra lỗi đường dẫn backend – Thử nghiệm tính năng bot bên frontend – Use case (share) – Thiết lập CI/CD (build, push, test) – Viết báo cáo	100%
2	Bùi Hoàng Cung	2310376	.	– Yêu cầu phi chức năng (NF) – Use case (upload) – Xử lý request gửi BE – Thử nghiệm tính năng bot bên frontend	100%
3	Nguyễn Trí Thành	2313127	L02	– NF – Use case (revoke) – Service xử lý BE API – Thử nghiệm tính năng bot bên frontend	100%
4	Võ Hùng Dũng	2310577	.	– NF – Use case (download) – Router chuyển lệnh về hàm xử lý – Thử nghiệm tính năng bot bên frontend	100%



Mục lục

1	Giới thiệu đề tài	3
1.1	Nội dung	3
1.2	Mục tiêu đề ra	3
1.3	Triển khai và cài đặt	4
2	Tổng quan hệ thống	5
3	Quá trình triển khai dự án (Frontend – Telegram Bot)	6
3.1	Giai đoạn 1: Thiết kế giao diện và phát triển nền tảng (04/11 – 20/11) .	6
3.1.1	Thiết kế kiến trúc Frontend Bot	6
3.1.2	Thiết kế luồng hội thoại (Conversation Flow)	6
3.1.3	Thiết kế UI/UX cho Telegram Bot	7
3.1.4	Chuẩn bị tài liệu và phân chia công việc	18
3.2	Giai đoạn 2: Hoàn thiện frontend và triển khai (20/11 – 06/12)	19
3.2.1	Triển khai logic Telegram Bot	19
3.2.2	Tích hợp Backend API	19
3.2.3	Container hóa Telegram Bot	19
3.2.4	Thiết lập CI/CD và môi trường chạy	19
3.3	Giai đoạn 3: Kiểm thử, demo và hoàn thiện báo cáo (06/12 – 12/12) . . .	19
3.3.1	Kiểm thử Telegram Bot	19
3.3.2	Chuẩn bị demo hệ thống	19
3.3.3	Hoàn thiện báo cáo đồ án	20

Tóm tắt nội dung

Báo cáo này trình bày quá trình thiết kế, xây dựng và triển khai một hệ thống chia sẻ file thông qua Telegram Bot, đóng vai trò như một frontend hội thoại cho người dùng cuối. Thay vì sử dụng giao diện web hoặc ứng dụng riêng biệt, hệ thống tận dụng nền tảng Telegram để cung cấp trải nghiệm tương tác đơn giản, trực quan và quen thuộc.

Telegram Bot chịu trách nhiệm tiếp nhận yêu cầu từ người dùng, xử lý luồng hội thoại và gọi các backend API thông qua giao thức HTTP(S). Backend đảm nhiệm các nghiệp vụ chính như lưu trữ thông tin file, tạo liên kết chia sẻ, xác thực quyền truy cập và cung cấp cơ chế tải file an toàn. Toàn bộ hệ thống được thiết kế theo hướng phân tầng, dễ mở rộng và thuận tiện cho việc triển khai.

Ngoài việc tập trung vào chức năng, báo cáo còn mô tả chi tiết kiến trúc hệ thống, thiết kế luồng hội thoại, cách cấu hình môi trường, cũng như quy trình triển khai và vận hành bằng Docker. Kết quả cho thấy hệ thống hoạt động ổn định, đáp ứng được các yêu cầu đề ra và có tiềm năng mở rộng trong các bài toán thực tế.

1 Giới thiệu đề tài

1.1 Nội dung

Trong những năm gần đây, các nền tảng nhắn tin tức thời như Telegram, Messenger hay Slack đã trở thành công cụ giao tiếp phổ biến trong đời sống và công việc. Bên cạnh chức năng nhắn tin, Telegram còn cung cấp Telegram Bot API, cho phép xây dựng các ứng dụng tương tác trực tiếp với người dùng thông qua giao diện hội thoại. Điều này mở ra nhiều cơ hội để phát triển các hệ thống dịch vụ mà không cần xây dựng giao diện đồ họa truyền thống.

Đề án này tập trung xây dựng một hệ thống chia sẻ file, trong đó Telegram Bot đóng vai trò là frontend chính. Người dùng có thể thực hiện các thao tác như tải file lên, xem danh sách file, tạo liên kết chia sẻ và tải file được chia sẻ trực tiếp thông qua Telegram. Các nghiệp vụ xử lý phức tạp được tách sang backend API, giúp frontend bot nhẹ, dễ bảo trì và dễ mở rộng.

1.2 Mục tiêu đề ra

Mục tiêu chính của đề án bao gồm:

- Xây dựng một Telegram Bot hoạt động ổn định, thân thiện với người dùng.
- Thiết kế luồng hội thoại rõ ràng, dễ sử dụng cho các chức năng liên quan đến file.
- Tích hợp bot với backend thông qua các API REST chuẩn.
- Triển khai hệ thống bằng Docker nhằm đảm bảo tính nhất quán của môi trường chạy.

Báo cáo này trình bày tổng quan hệ thống từ khâu phân tích yêu cầu, thiết kế, triển khai cho đến đánh giá kết quả đạt được.

1.3 Triển khai và cài đặt

Để triển khai và chạy hệ thống, chúng ta cần chuẩn bị các thành phần sau:

Đối với frontend:

- **Ngôn ngữ:** Go v1.25
- **Hệ thống quản lý:** Git

Đối với môi trường triển khai:

- **Container hóa:** Docker, Docker Compose
- **CI/CD:** GitHub Actions

Ngoài ra, chúng ta không thể thiếu thành phần này:

- **Telegram Bot Token:** Tạo từ BotFather trên Telegram Web
- **Đường dẫn của Backend API sử dụng cho mục đích chạy Telegram Bot:**
<https://file-sharing-v1-holy-field-13.fly.dev>
- **Đặc tả của Backend API:** Một trong hai đường dẫn sau:
 1. <https://file-sharing-v1-holy-field-13.fly.dev/openapi.yaml>
 2. <https://github.com/dath-251-thuanle/tele-file-sharing-be/blob/dev/api/openapi.yaml>

2 Tổng quan hệ thống

Hệ thống chia sẻ file qua Telegram Bot được thiết kế theo mô hình client-server, trong đó Telegram Bot đóng vai trò là client (frontend) và backend API đóng vai trò là server xử lý nghiệp vụ. Cách tiếp cận này giúp phân tách rõ ràng trách nhiệm của từng thành phần, đồng thời tạo điều kiện thuận lợi cho việc phát triển song song giữa frontend và backend.

Các thành phần chính của hệ thống bao gồm:

- **Telegram Bot (Frontend):** Được phát triển bằng ngôn ngữ Go, bot tiếp nhận các tương tác từ người dùng, xử lý luồng hội thoại và hiển thị kết quả phản hồi. Bot không lưu trữ dữ liệu lâu dài mà chủ yếu đóng vai trò trung gian.
- **Backend API:** Cung cấp các endpoint REST để xử lý nghiệp vụ như tạo file, quản lý file, tạo và xác thực liên kết chia sẻ. Backend xác thực người dùng dựa trên thông tin Telegram được gửi kèm trong header của mỗi request.
- **Hạ tầng triển khai:** Sử dụng Docker và Docker Compose để đóng gói và chạy bot trong môi trường container, giúp đảm bảo tính nhất quán giữa các môi trường phát triển và triển khai.

Bot giao tiếp với backend thông qua giao thức HTTP(S), sử dụng JSON làm định dạng trao đổi dữ liệu. Cách thiết kế này giúp hệ thống dễ dàng mở rộng hoặc tích hợp thêm các client khác trong tương lai.

3 Quá trình triển khai dự án (Frontend – Telegram Bot)

3.1 Giai đoạn 1: Thiết kế giao diện và phát triển nền tảng (04/11 – 20/11)

3.1.1 Thiết kế kiến trúc Frontend Bot

Trong giai đoạn đầu, nhóm xác định Telegram Bot đóng vai trò là lớp frontend tương tác trực tiếp với người dùng, hoạt động như một tầng trung gian giữa người dùng Telegram và hệ thống Backend API. Bot chịu trách nhiệm tiếp nhận yêu cầu từ người dùng, xử lý logic hội thoại và chuyển tiếp các yêu cầu nghiệp vụ đến backend thông qua các REST API đã được định nghĩa.

Nhóm đề xuất kiến trúc frontend theo mô hình phân tầng nhằm đảm bảo tính rõ ràng, dễ mở rộng và dễ bảo trì, bao gồm:

- **Router:** tiếp nhận và điều hướng message, command và callback query từ Telegram đến các handler tương ứng.
- **Handler:** xử lý logic hội thoại, command và callback của người dùng.
- **Service:** xử lý nghiệp vụ phía frontend như upload file, chia sẻ file và quản lý thông tin người dùng.
- **API Client:** chịu trách nhiệm giao tiếp với Backend thông qua các REST API.

Kiến trúc này giúp tách biệt rõ ràng giữa frontend và backend, đồng thời tạo điều kiện thuận lợi cho việc kiểm thử và mở rộng chức năng trong tương lai.

3.1.2 Thiết kế luồng hội thoại (Conversation Flow)

Nhóm tiến hành phân tích các use case của người dùng Telegram, bao gồm: tải lên file, xem danh sách file, tạo liên kết chia sẻ, thu hồi liên kết chia sẻ, xem metadata của file hoặc link chia sẻ, xác thực mật khẩu khi tải file và tải file về. Tuy nhiên, nhóm sẽ chú trọng vào 4 use case chính là:

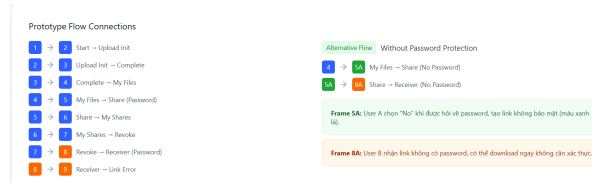
- **Upload:** Tải lên file
- **Share:** Tạo liên kết chia sẻ
- **Revoke:** Thu hồi liên kết chia sẻ
- **Download:** Tải file về

Dựa trên các use case này, nhóm xây dựng mock conversation flow cho từng chức năng, bao gồm:

- Luồng dựa trên command (ví dụ: `/start`, `/upload`, `/share`, ...).
- Luồng tương tác thông qua bàn phím trực tuyến (inline keyboard) và hàm gọi lại (callback query).

- Luồng nhập liệu theo từng bước (wizard flow) đối với các chức năng yêu cầu nhiều thông tin đầu vào.

Các luồng hội thoại được thiết kế nhằm đảm bảo tính trực quan, dễ sử dụng và phù hợp với đặc thù giao tiếp của nền tảng Telegram.



Hình 3.1.2.1: Mock conversational flow

3.1.3 Thiết kế UI/UX cho Telegram Bot

Nhóm tiến hành thiết kế giao diện người dùng cho Telegram Bot theo hai mức độ:

- **Low-fidelity mockup:** Xác định bố cục và nội dung cơ bản cho các màn hình như menu chính, danh sách file, danh sách link chia sẻ, form nhập mật khẩu và tính năng thu hồi link.

1. Flow /start

- User: Nhập /start (hoặc /me)
- FE: Gọi API GET /api/me
- Bot: Gửi lời chào và danh sách các lệnh

2. Flow /upload

- User: Nhập /upload
- FE: Gọi API POST /v1/files
- BE: Tạo presigned URL
- Bot: Nhận presigned URL và trả về format upload (hiệu lực 15 phút, kích thước tối đa 100MB, hỗ trợ các định dạng pdf, doc, png, ...)

3. Flow /upload complete

- Bot: Hiển thị trạng thái đang kiểm tra file
- Trường hợp:
 - * File đúng format → Bot hiển thị file hợp lệ
 - * File sai format → Bot yêu cầu upload file khác
- User: Nhập /upload complete
- FE: Gọi API POST /v1/files/:id/complete

- Bot: Thông báo upload thành công và gợi ý thao tác tiếp theo (ví dụ: `/myfiles`, `/share`)

4. Flow `/files`

- User: Nhập `/files`
- FE: Gọi API `GET /v1/files`
- Bot: Nhận danh sách file và hiển thị cho người dùng

5. Flow `/shares`

- User: Nhập `/shares`
- Trường hợp file đã tồn tại:
 - (a) User: Chọn file cần chia sẻ và nhập mật khẩu (nếu có)
 - (b) Bot: Gửi file và metadata cho BE
 - (c) BE: Gọi API `POST /v1/shares` và trả về link chia sẻ
 - (d) Bot: Gửi link cho User
 - (e) User: Chia sẻ link cho người khác
- Trường hợp chưa có file:
 - (a) User: Gửi file cần chia sẻ
 - (b) Bot: Kiểm tra file và phản hồi kết quả
 - (c) Bot: Gửi file và metadata cho BE nếu hợp lệ
 - (d) BE: Gọi API `POST /v1/shares` và trả về link chia sẻ
 - (e) Bot: Gửi link cho User

6. Flow `/shares/:id`

- User: Nhập `/shares/:id` (hoặc `/shares`)
- FE: Gọi API `GET /v1/shares/:id` hoặc `GET /v1/shares`
- Bot: Hiển thị link chia sẻ hoặc danh sách các link đã chia sẻ
- Bot: Gợi ý thu hồi link bằng lệnh `/shares/:id/revoke` (hoặc `/revoke`)

7. Flow `/revoke`

- User: Nhập `/shares/:id/revoke` (hoặc `/revoke`)
- Bot: Yêu cầu người dùng gửi link cần thu hồi (nếu dùng `/revoke`)
- FE: Gọi API `POST /v1/shares/:id/revoke`

– Trường hợp:

- * Bình thường: User nhập mật khẩu (nếu có) → Bot thông báo thu hồi thành công
- * Link đã bị thu hồi: Bot thông báo link đã bị thu hồi
- * Link không hợp lệ: Bot thông báo link không hợp lệ hoặc người dùng không phải chủ sở hữu

8. Flow tải file (metadata)

– FE: Gọi API GET /v1/shares/:id/download

– Bot: Kiểm tra metadata

– Trường hợp:

- * Link đã bị thu hồi → Bot thông báo link không tồn tại
- * Link đã hết hạn → Bot thông báo link đã hết hạn
- * Link yêu cầu mật khẩu → Bot hiển thị metadata (tên file, dung lượng, trạng thái bảo mật)
- * Không yêu cầu mật khẩu:
 - FE: Gọi API GET /v1/shares/:id/download để lấy presigned URL
 - Bot: Hiển thị link tải cho User

9. Flow nhập mật khẩu và tải file

– Bot: Yêu cầu người dùng nhập mật khẩu

– Trường hợp đúng:

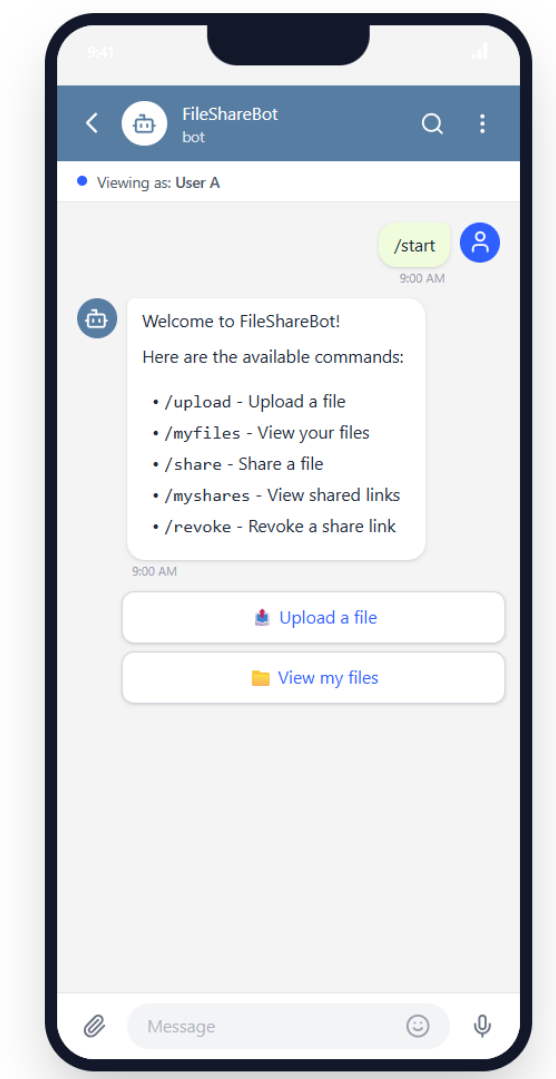
- * FE: Gọi API GET /v1/shares/:id/download để lấy presigned URL
- * Bot: Hiển thị link tải cho User

– Trường hợp sai:

- * Bot: Thông báo mật khẩu không đúng, còn tối đa 3 lần thử
- * Sau 3 lần sai: Bot yêu cầu chờ một khoảng thời gian (5 phút, 10 phút, ...)

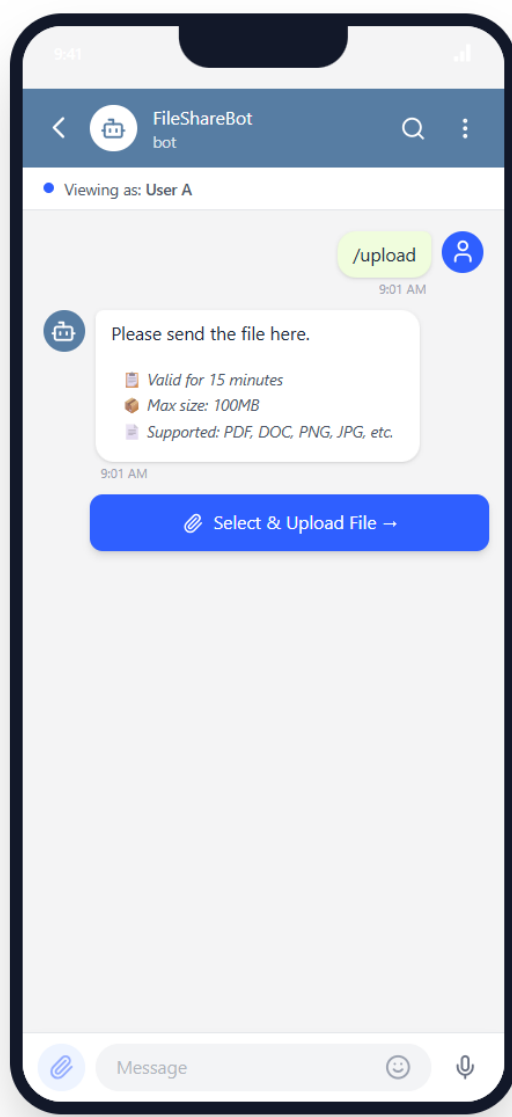
- **High-fidelity prototype:** Sử dụng công cụ tương tự với Figma để mô phỏng chi tiết thứ tự tương tác, nội dung tin nhắn, các nút bấm và callback tương ứng.

1. Flow /start



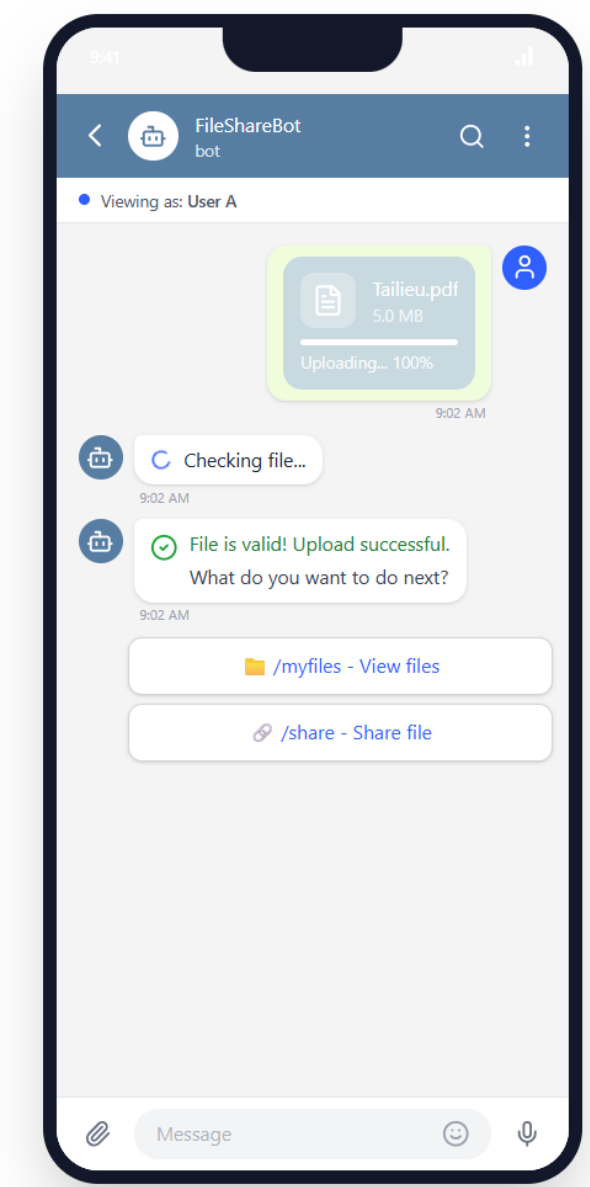
Hình 3.1.3.1: Flow /start

2. Flow /upload



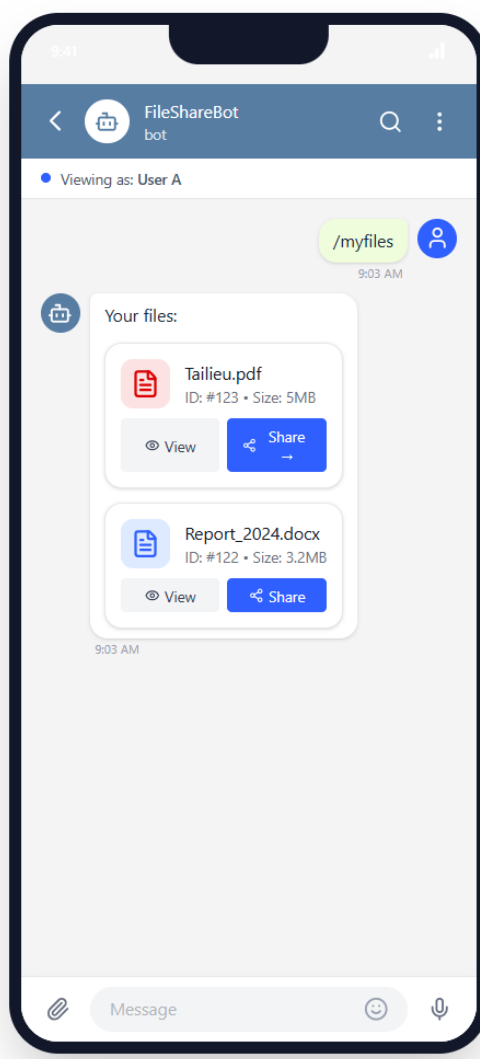
Hình 3.1.3.2: Flow /upload

3. Flow /upload complete



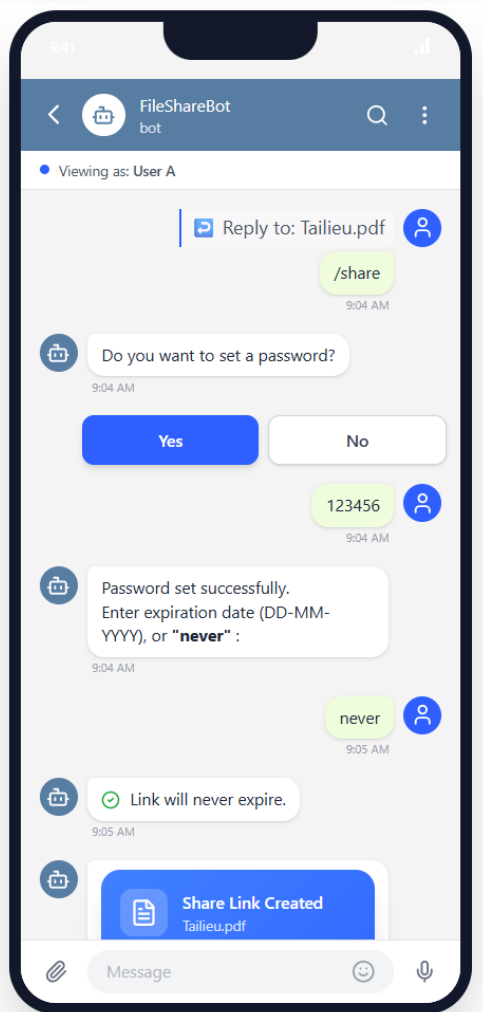
Hình 3.1.3.3: Flow /upload complete

4. Flow /files

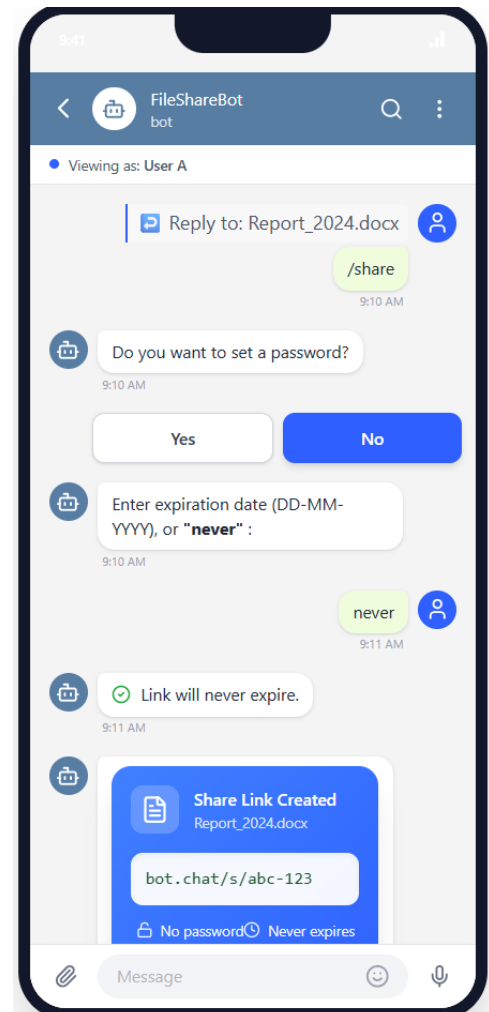


Hình 3.1.3.4: Flow /files

5. Flow /shares



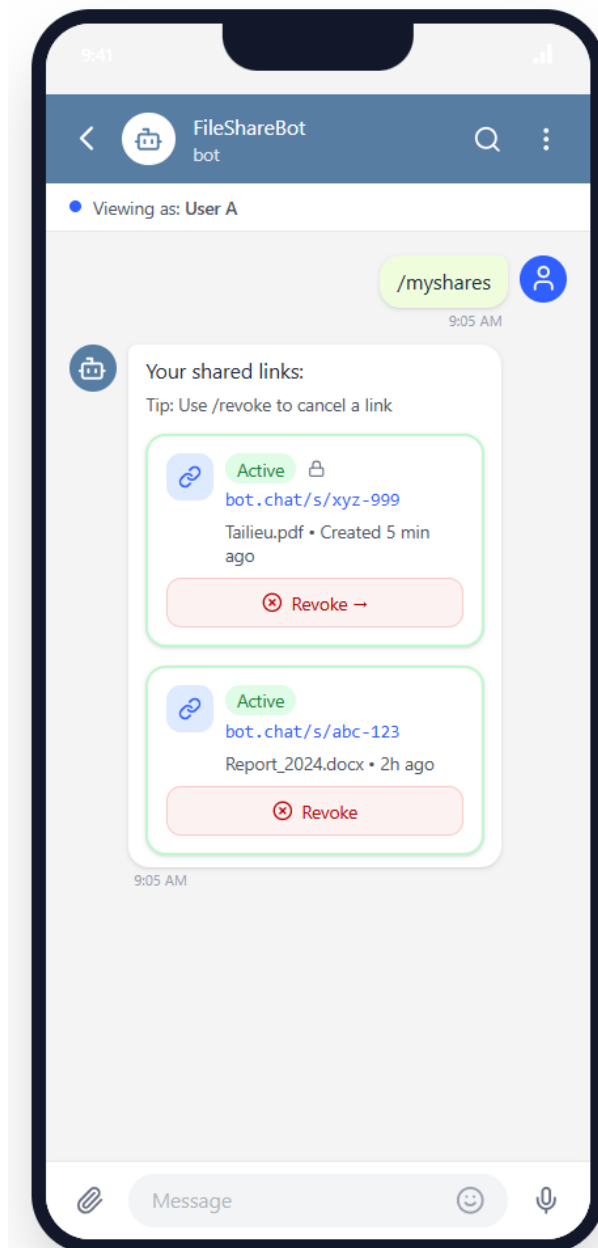
(a) Flow `/shares` có đặt mật khẩu



(b) Flow `/shares` không có đặt mật khẩu

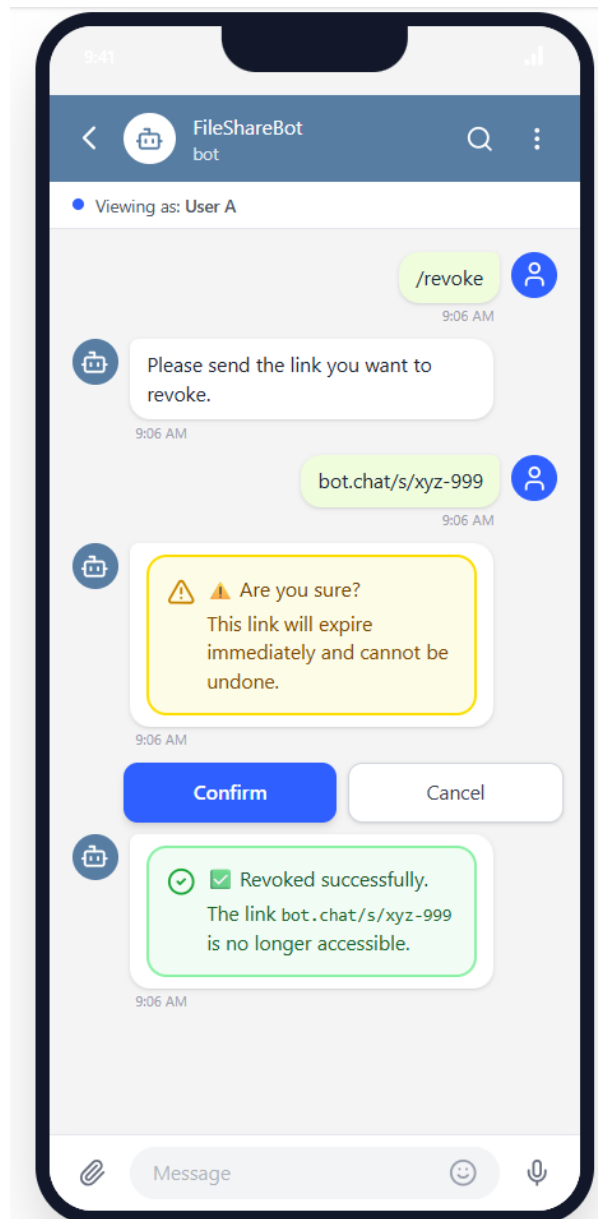
Hình 3.1.3.5: Flow `/share` của Telegram Bot

6. Flow `/shares/:id`



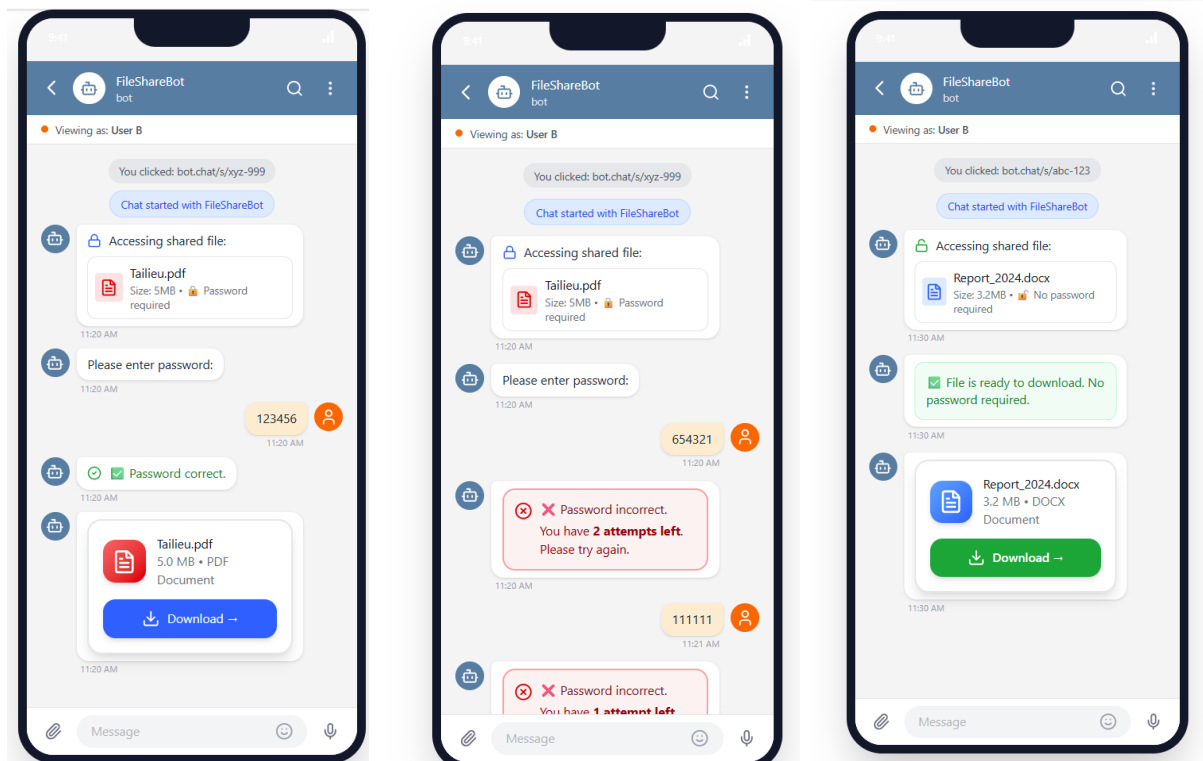
Hình 3.1.3.6: Flow `/shares/:id`

7. Flow /revoke



Hình 3.1.3.7: Flow /revoke

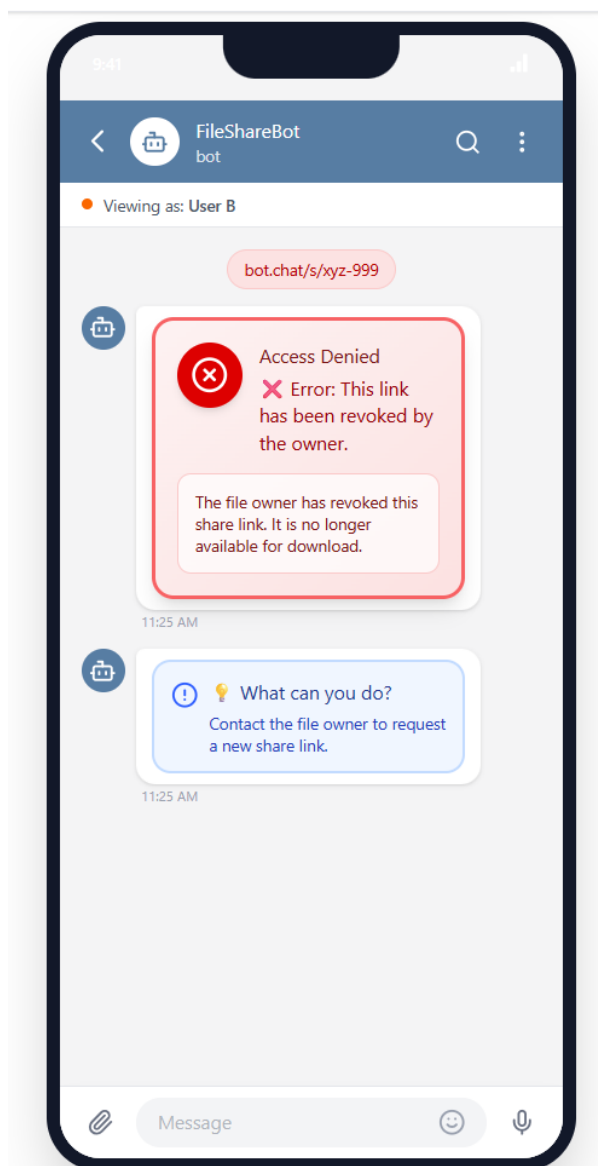
8. Flow tải file (metadata)



(a) Flow tải file có mật khẩu và nhập đúng (b) Flow tải file có mật khẩu và nhập sai (c) Flow tải file không có mật khẩu

Hình 3.1.3.8: Flow tải file (metadata) của Telegram Bot

9. Flow nhập mật khẩu và tải file



Hình 3.1.3.9: Flow nhập mật khẩu và tải file

Ngoài ra, nhóm cũng chuẩn hóa nội dung hiển thị như văn bản, emoji và thông báo lỗi nhằm nâng cao trải nghiệm người dùng và đảm bảo tính nhất quán trong toàn bộ hệ thống.

3.1.4 Chuẩn bị tài liệu và phân chia công việc

Song song với quá trình thiết kế, nhóm xây dựng danh sách các command và callback event của bot, đồng thời thống nhất cách frontend gọi API dựa trên tài liệu OpenAPI do backend cung cấp. Các công việc được phân chia rõ ràng cho từng thành viên phụ trách các mảng như router, handler, service và DevOps.

3.2 Giai đoạn 2: Hoàn thiện frontend và triển khai (20/11 – 06/12)

3.2.1 Triển khai logic Telegram Bot

Nhóm tiến hành cài đặt router để điều hướng message, command và callback query từ Telegram. Các handler được xây dựng tương ứng với từng chức năng chính như tải lên file, chia sẻ file, quản lý file và quản lý thông tin người dùng.

3.2.2 Tích hợp Backend API

Frontend bot được tích hợp với backend thông qua module `api/client.go`, chịu trách nhiệm gọi các endpoint như upload file, báo cáo hoàn tất upload, quản lý file, tạo và thu hồi link chia sẻ, lấy metadata và tải file. Các request được chuẩn hóa header để truyền thông tin người dùng Telegram, đồng thời các lỗi HTTP từ backend được xử lý và chuyển thành thông báo thân thiện cho người dùng.

3.2.3 Container hóa Telegram Bot

Nhóm xây dựng Dockerfile theo mô hình multi-stage build cho ứng dụng Go nhằm tối ưu kích thước image và tăng cường bảo mật thông qua việc chạy ứng dụng với non-root user. File `docker-compose.yml` được tạo để hỗ trợ môi trường phát triển, cho phép load biến môi trường từ file cấu hình, tự động restart bot và dễ dàng chạy demo chỉ với một lệnh.

3.2.4 Thiết lập CI/CD và môi trường chạy

GitHub Actions được thiết lập để tự động build Telegram Bot và kiểm tra định dạng mã nguồn. Các biến môi trường quan trọng như Bot Token, Backend API Base URL và chế độ debug được chuẩn hóa. Nhóm cũng xây dựng tài liệu hướng dẫn deploy bot.

3.3 Giai đoạn 3: Kiểm thử, demo và hoàn thiện báo cáo (06/12 – 12/12)

3.3.1 Kiểm thử Telegram Bot

Nhóm thực hiện kiểm thử thủ công toàn bộ các luồng hội thoại chính, bao gồm upload file với nhiều định dạng khác nhau, tạo và thu hồi link chia sẻ, cũng như xác thực mật khẩu khi tải file. Ngoài ra, các tình huống lỗi như lỗi API, dữ liệu nhập không hợp lệ và upload thất bại cũng được kiểm tra nhằm đảm bảo hệ thống hoạt động ổn định.

3.3.2 Chuẩn bị demo hệ thống

Telegram Bot được chạy trong môi trường Docker để đảm bảo tính ổn định trong quá trình demo. Nhóm xây dựng kịch bản demo bao gồm các bước: người dùng upload file, tạo link chia sẻ và người dùng khác tải file thông qua link chia sẻ. Kết quả chạy thực tế được ghi nhận để minh họa cho báo cáo.



3.3.3 Hoàn thiện báo cáo đồ án

Cuối cùng, nhóm tổng hợp toàn bộ nội dung liên quan đến kiến trúc frontend, luồng hội thoại và quá trình triển khai hệ thống. Báo cáo cũng đánh giá các kết quả đạt được, những hạn chế còn tồn tại và đề xuất các hướng cải tiến trong tương lai như triển khai webhook mode, caching và phân tích hành vi người dùng.